

CAHIER DES CHARGES

Projet de Fin d'Annee (PFA)

FreelanceStudent

Plateforme de Freelancing Academique pour Etudiants

Etablissement	EMSI Casablanca — Ecole Marocaine des Sciences de l'Ingenieur
Filiere	3IIR — Ingenierie Informatique et Reseaux
Semestre	S2 — Annee 2025/2026
Projet	FreelanceStudent — Plateforme de freelancing pour etudiants
Technologies	Django (Python) — PostgreSQL — HTML/CSS/JS — REST API
Date de redaction	Mars 2026
Version	1.0
Binome	NAHAIRY ZAKARIA HATIM Amine EZZOUINE

1. Contexte et Objectifs du Projet

1.1 Contexte General

Dans le contexte economique actuel, le freelancing connait une croissance exponentielle a l'echelle mondiale. Les entreprises cherchent de plus en plus a collaborer avec des talents independants pour des missions courtes et specifiques. Parallelement, les etudiants en informatique, design, marketing ou gestion possedent des competences reelles et recherchent des opportunités de mettre en pratique leurs connaissances, de constituer un portfolio professionnel et de generer des revenus complementaires.

Cependant, il n'existe pas de plateforme dediee specifiquement aux etudiants marocains, adaptee a leurs contraintes (disponibilite limitee, pas de statut juridique d'auto-entrepreneur, besoin de credits academiques). Les plateformes existantes comme Upwork ou Fiverr sont generiques, en anglais, et ne sont pas adaptees au contexte academique local.

1.2 Problematique

Comment permettre aux étudiants de valoriser leurs compétences auprès de clients réels (entreprises, startups, particuliers), tout en s'inscrivant dans un cadre académique sécurisé, transparent et adapté à leur disponibilité, via une plateforme numérique dédiée ?

1.3 Objectifs du Projet

Objectifs généraux :

- Créer une plateforme web permettant aux étudiants de proposer leurs services en tant que freelancers.
- Permettre aux clients de publier des missions et de recruter des étudiants qualifiés.
- Offrir un environnement sécurisé, transparent et adapté au contexte académique marocain.

Objectifs spécifiques :

- Implémenter un système de profils étudiants avec portfolio et compétences.
- Développer un moteur de recherche et de mise en relation client/étudiant.
- Intégrer un système de suivi des missions et de notation mutuelle.
- Assurer la vérification du statut étudiant via la carte étudiante.
- Mettre en place un back-office d'administration pour la modération.

2. Perimetre du Projet

2.1 Ce qui est Inclus

- Inscription et authentification des étudiants (avec vérification du statut étudiant).
- Inscription et authentification des clients (entreprises ou particuliers).
- Profil étudiant : photo, bio, compétences, portfolio de projets.
- Publication de missions par les clients (titre, description, budget, délai, catégorie).
- Système de candidature : les étudiants postulent aux missions.
- Messagerie interne entre étudiant et client pour chaque mission.
- Tableau de bord étudiant : mes missions, mes candidatures, mes gains.
- Tableau de bord client : mes offres publiées, candidatures reçues, missions en cours.
- Système de notation et d'avis après completion d'une mission.
- Panneau d'administration : gestion des utilisateurs, modération des contenus.
- Interface responsive utilisable sur desktop et mobile.

2.2 Ce qui est Exclu (Limites PFA)

- Systeme de paiement en ligne integre (ex : CMI, PayPal) — le paiement est gere hors plateforme dans un premier temps.
- Application mobile native (iOS / Android) — uniquement web responsive.
- Intelligence artificielle pour la recommandation automatique.
- Verification electronique automatique des cartes etudiantes — la verification est manuelle par l'administrateur.
- Multi-langue complet (arabe, anglais) — la plateforme sera en francais uniquement.
- Gestion comptable et fiscale des revenus des etudiants.

3. Acteurs et Besoins Utilisateurs

3.1 Acteurs du Systeme

Acteur	Role et responsabilites
Etudiant Freelancer	S'inscrit, cree son profil/portfolio, postule aux missions, communique avec les clients, livre les travaux, recoit une notation.
Client	Entreprise ou particulier qui s'inscrit, publie des missions, selectionne des candidats, suit l'avancement, evalue l'etudiant.
Administrateur	Modere les contenus, verifie les comptes etudiants, gere les categories de missions, resout les litiges, genere des statistiques.

3.2 Besoins par Acteur

Etudiant Freelancer :

- Pouvoir creer un profil professionnel mettant en valeur ses competences et projets.
- Consulter et filtrer les missions disponibles par categorie, budget et duree.
- Postuler avec une lettre de motivation et son profil en un clic.
- Suivre l'etat de ses candidatures (en attente, acceptee, refusee).
- Echanger avec le client via la messagerie integree.
- Consulter ses statistiques : missions realisees, note moyenne, revenus.

Client :

- Creer un compte entreprise ou particulier avec verification de l'identite.
- Publier une mission en precisant le budget, les competences requises et le delai.
- Consulter les profils des candidats et leurs portfolios.
- Selectionner un etudiant et declencher la mission.
- Communiquer avec l'etudiant et suivre l'avancement.

- Laisser une note et un commentaire a la fin de la mission.

Administrateur :

- Valider ou rejeter les inscriptions etudiantes apres verification du statut.
- Moderer les missions et profils signales.
- Gerer les categories de competences et de missions.
- Acceder aux statistiques globales de la plateforme.

4. Exigences Fonctionnelles

4.1 Fonctions Principales

Ref.	Description de la fonctionnalite
F1	Inscription et connexion securisee pour etudiants, clients et administrateur (JWT ou sessions Django).
F2	Verification du statut etudiant : upload de la carte etudiante, validation manuelle par l'admin.
F3	Gestion du profil etudiant : photo, bio, competences (tags), liens GitHub/LinkedIn, portfolio de projets.
F4	Publication de mission par un client : titre, description, budget (MAD), competences requises, categorie, delai.
F5	Moteur de recherche et filtrage des missions : par categorie, budget min/max, duree, mots-cles.
F6	Systeme de candidature : postuler a une mission avec lettre de motivation, voir les candidats pour le client.
F7	Messagerie interne contextualisee par mission (fil de discussion etudiant/client).
F8	Tableau de bord etudiant : candidatures en cours, missions actives, historique, note moyenne.
F9	Tableau de bord client : missions publiees, candidatures recues, missions en cours, historique.
F10	Systeme de notation et avis bilateraux apres cloture de mission (1 a 5 etoiles + commentaire).
F11	Panneau d'administration : CRUD utilisateurs, moderation missions/profils, statistiques.
F12	Notifications (in-app) : nouvelle candidature, message recu, mission acceptee/terminee.

4.2 Scenario d'Utilisation Principal

Scenario : Un client publie une mission de creation d'une interface web.

1. Le client se connecte, clique sur « Publier une mission » et remplit le formulaire (titre, description, budget 800 MAD, delai 2 semaines, competences : HTML/CSS/Django).

2. La mission apparait dans le fil des missions disponibles pour les etudiants correspondant au profil.
3. Un etudiant trouve la mission via le moteur de recherche, consulte les details et postule avec une lettre.
4. Le client consulte les candidatures, evalue les portfolios et selectionne l'etudiant.
5. La mission passe en statut « En cours ». Echanges via la messagerie, livraison du travail.
6. Le client valide le livrable, cloture la mission et laisse une note. L'etudiant note le client en retour.

5. Exigences Non Fonctionnelles

5.1 Performance et Disponibilite

- Le chargement de la liste des missions (50 missions) doit etre inferieur a 2 secondes.
- La plateforme doit supporter au minimum 200 utilisateurs simultanement en phase PFA.
- La messagerie doit afficher les nouveaux messages en moins de 3 secondes sans rechargement de page.

5.2 Securite et Confidentialite

- Authentification par sessions Django securisees (CSRF protection, cookies HttpOnly).
- Mots de passe stockes avec hachage bcrypt/Argon2 (via Django auth).
- Acces aux donnees strictement limite selon le role (etudiant, client, admin).
- Uploads de fichiers (carte etudiante, portfolio) stockes hors racine web.
- Protection contre les injections SQL et XSS (ORM Django, templates auto-escapes).

5.3 Ergonomie et Accessibilite

- Interface responsive : utilisable sur ordinateur, tablette et smartphone.
- Navigation intuitive : acces aux fonctions principales en moins de 3 clics.
- Formulaire avec validation cote client et cote serveur avec messages d'erreur explicites.
- Design clair, moderne et professionnel, adapte a un public etudiant et entreprise.

5.4 Contraintes Techniques

- Le systeme doit etre deployable sur un serveur VPS standard ou en local (developement).
- La base de donnees doit permettre les migrations sans perte de donnees (Django migrations).
- Le code doit etre versionne sur GitHub avec des commits reguliers et documentaires.

6. Environnement Technique

6.1 Plateforme Cible

Application web responsive accessible depuis un navigateur moderne (Chrome, Firefox, Edge). Developpement en local avec Django's built-in development server, deploiement possible sur serveur VPS Linux (Ubuntu) avec Gunicorn + Nginx.

6.2 Technologies Envisagees

Composant	Technologie choisie et justification
Backend	Django 4.x (Python) — Framework robuste, securise, avec ORM puissant. Ideal pour gerer l'authentification, les modeles de donnees et les APIs.
Frontend	HTML5 / CSS3 / JavaScript — Templates Django (Jinja2). Possiblement Bootstrap 5 pour le design responsive.
Base de donnees	PostgreSQL — Base relationnelle performante, bien integree avec Django via django-postgres.
API REST	Django REST Framework (DRF) — Pour les endpoints JSON consommes par le frontend (notifications, messagerie).
Authentification	Django Authentication System — Sessions + tokens CSRF. Possible extension via django-allauth.
Stockage fichiers	Django FileField avec gestion locale en dev, compatible S3 en production.
Versioning	Git + GitHub — Pour le suivi du code source et la collaboration en equipe.
Deploiement	Gunicorn + Nginx ou juste vercel.app (optionnel en PFA) ou serveur de dev Django.

6.3 Architecture Generale

L'application suit le patron MVT (Model-View-Template) de Django :

- Models : User, StudentProfile, ClientProfile, Mission, Application, Message, Review.
- Views : vues basees sur des classes (CBV) pour les CRUD, vues fonctionnelles pour la logique metier.
- Templates : pages HTML rendues cote serveur avec les donnees du contexte Django.
- URLs : routage claire par module (accounts/, missions/, dashboard/, admin/).

7. Planning Previsionnel et Livrables

7.1 Planning Global

Periode	Activites
Semaine 1–2	Redaction du CDC, etat de l'art (plateformes freelancing existantes), conception de la base de donnees (diagramme ER).
Semaine 3–4	Mise en place du projet Django, modeles de donnees, migrations, maquettes des ecrans principaux (Figma ou papier).
Semaine 5–6	Developpement : authentication, profils etudiants/clients, publication et recherche de missions.
Semaine 7–8	Developpement : systeme de candidature, messagerie interne, tableaux de bord.
Semaine 9	Developpement : notation/avis, panneau d'administration, notifications.
Semaine 10	Tests fonctionnels, corrections de bugs, integration finale, demo.
Semaine 11	Redaction du rapport PFA, preparation de la soutenance et des slides.

7.2 Livrables Attendus

- CDC (ce document) — version finale validee par l'encadrant.
- Diagramme entite-relation (ER) de la base de donnees.
- Maquettes des ecrans principaux (wireframes).
- Code source versionne sur GitHub (depot public ou prive partage avec l'encadrant).
- Application web fonctionnelle et demonstrable en local.
- Rapport PFA : etat de l'art, architecture, implementation, tests, conclusion.
- Slides de soutenance (presentation orale).

7.3 Criteres de Succes

- Les 3 types d'utilisateurs (etudiant, client, admin) peuvent accomplir leurs flux principaux sans erreur.
- Au moins 5 missions fictives creees, 3 candidatures et 1 cycle complet (publication, candidature, selection, notation) demonstre.
- Code source propre, versionne, avec un README de lancement du projet.
- Interface responsive fonctionnelle sur desktop et mobile.